EX NO 7

DATE

AIM

we used logistic regression to predict the probability of default using income and balance on the Default data set. We will now estimate the test error of this logistic regression model using the validation set approach. Do not forget to set a random seed before beginning your analysis.

(a) Fit a logistic regression model that uses income and balance to predict default.

CODE

```
X = sm.add_constant(Default[['income', 'balance']])
fit = sm.GLM(Default['default_yes'], X,
family=sm.families.Binomial()).fit()
summarize(fit)
```

OUTPUT

	coef	std err	z	P>|z|
const	-11.540500	0.435000	-26.544	0.0
income	0.000021	0.000005	4.174	0.0
balance	0.005600	0.000000	24.835	0.0

OBSERVATION

Balance is highly significant (very small p-value). income is not significant. Higher balance means higher probability of default.

(b) Using the validation set approach, estimate the test error of this model. In order to do this, you must perform the following steps:

i. Split the sample set into a training set and a validation set.

ii. Fit a multiple logistic regression model using only the training observations.

iii. Obtain a prediction of default status for each individual in the validation set by computing the posterior probability of default for that individual, and classifying the individual to the default category if the posterior probability is greater than 0.5.

iv. Compute the validation set error, which is the fraction of the observations in the validation set that are misclassified.

CODE

```
def val_error(features, seed):
    tr, va = train_test_split(Default, test_size=0.5, random_state=seed)
    Xtr = sm.add_constant(tr[features])
    Xva = sm.add_constant(va[features])
    m = sm.GLM(tr['default_yes'], Xtr,
family=sm.families.Binomial()).fit()
    pred = (m.predict(Xva) > 0.5).astype(int)
    return np.mean(pred != va['default_yes'])

print(val_error(['income', 'balance'], seed=1))
```

OUTPUT

```
0.025
```

OBSERVERATION

The validation set error is about 2.5-2.7%. Only about 3.3% of people default, so this is a low error, although always predicting “No” would also give about 3.3%.

(c) Repeat the process in (b) three times, using three different splits of the observations into a training set and a validation set. Comment on the results obtained.

CODE

```
for seed in [2, 3, 4]:
    print(seed, val_error(['income', 'balance'], seed))
```

OUTPUT

```
2 0.0248
3 0.0248
4 0.0238
```

OBSERVATION

The error changes from split to split (about 2.4% to 2.9%). So the validation set estimate is variable and depends on which observations are in the training set.

(d) Now consider a logistic regression model that predicts the probability of default using income, balance, and a dummy variable for student. Estimate the test error for this model using the validation set approach. Comment on whether or not including a dummy variable for student leads to a reduction in the test error rate.

CODE

```
for seed in [1, 2, 3, 4]:
    print(seed, val_error(['income', 'balance'], seed),
          val_error(['income', 'balance', 'student_yes'], seed))
```

OUTPUT

```
1 0.025 0.0262
2 0.0248 0.0254
3 0.0248 0.0252
4 0.0238 0.0244
```

OBSERVATION

- Adding student does not reduce the test error in a consistent way. The two errors are very close and sometimes one is lower, sometimes the other.
- So including the student dummy variable does not clearly improve prediction.

EX NO 8

DATE

AIM

we saw that the cross_validate() function can be used in order to compute the LOOCV test error estimate. Alternatively, one could compute those quantities using just sm.GLM() and the predict() method of the fitted model within a for loop. You will now take this approach in order to compute the LOOCV error for a simple logistic regression model on the Weekly data set. Recall that in the context of classification problems, the LOOCV error is given in (5.4).

(a) Fit a logistic regression model that predicts Direction using Lag1 and Lag2.

CODE

```
fit_a = sm.GLM(yw, Xw, family=sm.families.Binomial()).fit()
summarize(fit_a)
```

OUTPUT

	coef	std err	z	P>|z|
const	0.2212	0.061	3.599	0.000
Lag1	-0.0387	0.026	-1.477	0.140
Lag2	0.0602	0.027	2.270	0.023

OBSERVATION

Lag2 is slightly significant and Lag1 is not. Both effects are small.

(b) Fit a logistic regression model that predicts Direction using Lag1 and Lag2 ***using all but the first observation.***

CODE

```
fit_b = sm.GLM(yw.iloc[1:], Xw.iloc[1:],
family=sm.families.Binomial()).fit()
summarize(fit_b)
```

OUTPUT

	coef	std err	z	P>|z|
const	0.2232	0.061	3.630	0.000
Lag1	-0.0384	0.026	-1.466	0.143
Lag2	0.0608	0.027	2.291	0.022

OBSERVATION

The coefficients are almost the same as the full model, since removing one of 1089 observations changes very little.

(c) Use the model from (b) to predict the direction of the first observation. You can do this by predicting that the first observation will go up if $P(\text{Direction} = \text{"Up"}|\text{Lag1}, \text{Lag2}) > 0.5$. Was this observation correctly classified?

CODE

```
p1= fit_b.predict(Xw.iloc[[0]]).iloc[0]
print('prob Up:', p1, ' predicted:', int(p1 > 0.5), ' actual:',
yw.iloc[0])
```

OUTPUT

```
prob Up: 0.5713923205204429  predicted: 1  actual: 0
```

OBSERVATION

The model predicts “Up” (probability above 0.5), but the first week was actually “Down”, so it is **not** correctly classified.

(d) Write a for loop from $i = 1$ to $i = n$, where n is the number of observations in the data set, that performs each of the following

i. Fit a logistic regression model using all but the ith observation to predict Direction using Lag1 and Lag2.

ii. Compute the posterior probability of the market moving up for the ith observation.

iii. Use the posterior probability for the ith observation in order to predict whether or not the market moves up.

iv. Determine whether or not an error was made in predicting the direction for the ith observation. If an error was made, then indicate this as a 1, and otherwise indicate it as a 0.

CODE

```
n = len(W)
errors = np.zeros(n)
for i in range(n):
    keep = np.arange(n) != i
    m = sm.GLM(yw[keep], Xw[keep], family=sm.families.Binomial()).fit()
    p = m.predict(Xw.iloc[[i]]).iloc[0]
    errors[i] = int((p > 0.5) != yw.iloc[i])

    print('Loop finished, observations:', n)
print('First 10 errors:', errors[:10])
print('Total errors:', int(errors.sum()))
```

OUTPUT

```
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
Loop finished, observations: 1089
...
Loop finished, observations: 1089
Loop finished, observations: 1089
First 10 errors: [1. 1. 0. 1. 0. 1. 0. 0. 0. 1.]
Total errors: 490
```

OBSERVATION

Each loop leaves out one week, fits on the other 1088 weeks, predicts the left-out week, and stores 1 for an error and 0 otherwise.

(e) Take the average of the *n* numbers obtained in (d)iv in order to obtain the LOOCV estimate for the test error. Comment on the results.

CODE

```
print(errors.mean())
```

OUTPUT

```
0.44995408631772266
```

OBSERVATION

LOOCV error is about 0.45, so the model is right about 55% of the time. That is close to always predicting “Up” (about 56%), so the model has almost no predictive power.